

Instituto Tecnológico y de Estudios Superiores de Monterrey



Analítica de Datos y
Herramientas de Inteligencia Artificial

Reporte de actividad 3.2

Profesor: Alfredo García Suárez

Camila Trujillo Beristain | A01737170
Bernardo Quintana López | A01658064
Fernando Guadarrama González | A01379340
Mauricio Goris García | A01736428

Campus Puebla

30 de marzo de 2025

Reporte Explicativo del Análisis de Datos

El reporte documenta el análisis de un conjunto de datos relacionados con la interacción de usuarios en una plataforma de aprendizaje, en este caso WUUPI. El objetivo es detectar patrones de comportamiento, identificar datos faltantes o atípicos, transformar variables para facilitar su análisis y aplicar modelos de ajuste que permitan predecir tendencias individuales.

Comenzamos nuestro análisis importando las bibliotecas necesarias. En la **Celda 3** incluimos pandas y numpy para la manipulación de datos, matplotlib.pyplot y seaborn para la visualización, scipy.optimize.curve_fit para el modelado no lineal, y r2_score de sklearn para medir el ajuste de los modelos. Estas herramientas nos permitieron estructurar el trabajo de manera técnica y analítica desde el inicio.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import scipy.special as special
from scipy.optimize import curve_fit
import seaborn as sns
from sklearn.metrics import r2_score
```



Estas herramientas nos permiten trabajar con estructuras de datos, generar gráficas, ajustar modelos no lineales y evaluar la precisión de predicciones.

Cargamos el archivo DataAnalytics.csv utilizando pd.read_csv(). Este archivo contiene registros de múltiples variables relacionadas con el comportamiento de los usuarios en una plataforma educativa (WUUPI), incluyendo datos como el número de interacción, el tiempo invertido en cada fase, y la precisión en tareas ejecutadas. Almacenamos esta información en la variable data, que usamos como base para todo el análisis.

```
#Cargamos los datos
data = pd.read_csv('DataAnalytics.csv')
```



Evalúamos la calidad de los datos identificando los valores nulos con .isnull().sum(). Detectamos que varias columnas, particularmente las relacionadas con el tiempo, contenían datos faltantes. Después, dividimos el conjunto en variables numéricas (numericas) y cualitativas (cualitativas), y realizamos imputaciones de los valores faltantes usando la media de cada columna con .fillna(). Esta técnica nos permitió conservar los registros sin distorsionar en exceso las distribuciones originales.

```
valores_nulos = data.isnull().sum()
print(valores_nulos)
```

Python

Administrador	0
Usuario	0
botón correcto	762
tiempo de interacción	762
mini juego	156
número de interacción	762
color presionado	762
dificultad	0
fecha	0
Juego	0
auto push	762
tiempo de lección	177
tiempo de sesión	606

```
numericas = data.select_dtypes(include=['int64', 'float64'])
cualitativas = data.select_dtypes(include=['object'])

numericas_generales = numericas.drop(['tiempo de sesión', 'tiempo de lección'], axis=1)
numericas_generales_sin_nulos = numericas_generales.fillna(numericas_generales.mean())
```

Python

En la siguiente etapa, aislamos las variables tiempo de sesión y tiempo de lección, que son fundamentales para comprender la duración e intensidad de la participación del usuario. Les imputamos sus valores faltantes también con la media. Al tratar estas variables por separado, pudimos analizar patrones de comportamiento más detalladamente, ya que reflejan distintos momentos de interacción con el sistema.

```
tiempo_sesion = numericas['tiempo de sesión']
tiempo_leccion = numericas['tiempo de lección']

media_sesion_sin_ceros = tiempo_sesion.mean()
media_leccion_sin_ceros = tiempo_leccion.mean()

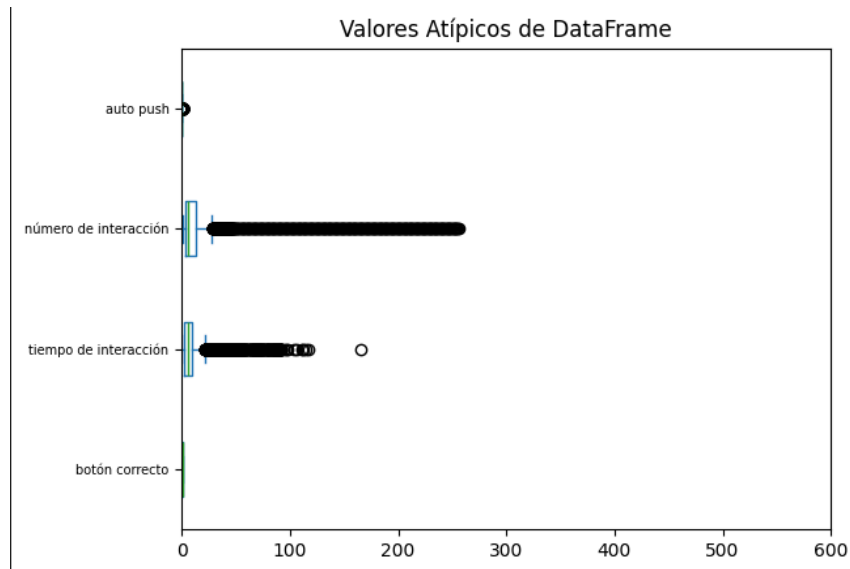
tiempo_sesion_sin_nulos = tiempo_sesion.fillna(media_sesion_sin_ceros)
● tiempo_leccion_sin_nulos = tiempo_leccion.fillna(media_leccion_sin_ceros)
```

Python

Generamos boxplots para observar la dispersión de las variables y detectar valores atípicos. Esto lo hicimos con `.plot(kind='box')`, ajustando el eje horizontal para facilitar la interpretación. Estos diagramas nos ayudaron a identificar usuarios con tiempos de sesión o interacción extremadamente altos, que podrían estar sesgando los resultados. A partir de estas observaciones, consideramos el posible tratamiento posterior de esos casos extremos.

```
fig = plt.figure(figsize = (15,10))
numericas_generales_sin_nulos.plot(kind='box', vert=False)
plt.xlim([0, 600])
plt.title('Valores Atípicos de DataFrame')
plt.yticks(fontsize=7, rotation=0)
plt.show()
```

Python



Estas gráficas mostraron la presencia de valores atípicos en variables como tiempo de interacción, lo cual permitió tomar decisiones más informadas sobre el tratamiento de datos extremos.

Creamos un nuevo DataFrame (numericas_con0) donde imputamos los valores faltantes con ceros en lugar de medias. Esta comparación visual nos permitió explorar cómo diferentes métodos de imputación afectan la distribución de los datos. Observar esta diferencia fue útil para evaluar qué técnica era más adecuada según el tipo de variable, dado que imputar con ceros puede subestimar el tiempo real invertido por los usuarios.

```
numericas_con0 = pd.concat([tiempo_sesion_sin_nulos, tiempo_leccion_sin_nulos], axis=1)
```

Python

Realizamos la conversión de los valores de texto en las columnas Usuario y Administrador a valores numéricos mediante el método `.replace()`. Esta transformación fue necesaria porque

muchas funciones de análisis requieren trabajar con datos numéricos. Al asignar números a cada nombre, preparamos los datos para filtrarlos y modelarlos de forma eficiente.

```
data_final.Usuario = data_final.Usuario.replace({'LEONARDO': '1'}, regex=False)
data_final.Usuario = data_final.Usuario.replace({'ALEIDA': '2'}, regex=False)
data_final.Usuario = data_final.Usuario.replace({'NICOLAS': '3'}, regex=False)
data_final.Usuario = data_final.Usuario.replace({'JOSE JAVIER ': '4'}, regex=False)
data_final.Usuario = data_final.Usuario.replace({'LEONARDO ': '5'}, regex=False)
data_final.Usuario = data_final.Usuario.replace({'JESUS ALEJANDRO': '6'}, regex=False)
data_final.Usuario = data_final.Usuario.replace({'ramiro isai': '7'}, regex=False)
data_final.Usuario = data_final.Usuario.replace({'ADRIAN ': '8'}, regex=False)
data_final.Usuario = data_final.Usuario.replace({'SERGIO ANGEL': '9'}, regex=False)
data_final.Usuario = data_final.Usuario.replace({'CARLOS ENRIQUE': '10'}, regex=False)
data_final.Usuario = data_final.Usuario.replace({'DENISSE': '11'}, regex=False)
data_final.Usuario = data_final.Usuario.replace({'Yael DAVID': '12'}, regex=False)
data_final.Usuario = data_final.Usuario.replace({'VALENTIN': '13'}, regex=False)
data_final.Usuario = data_final.Usuario.replace({'erick ': '14'}, regex=False)
data_final.Usuario = data_final.Usuario.replace({'IKER BENJAMIN': '15'}, regex=False)
data_final.Usuario = data_final.Usuario.replace({'NICOLAS |': '16'}, regex=False)
data_final.Usuario = data_final.Usuario.replace({'ERICK OSVALDO': '17'}, regex=False)
data_final.Usuario = data_final.Usuario.replace({'CONCEPCION': '18'}, regex=False)
data_final.Usuario = data_final.Usuario.replace({'KYTZIA': '19'}, regex=False)
data_final.Usuario = data_final.Usuario.replace({'AUSTIN': '20'}, regex=False)
data_final.Usuario = data_final.Usuario.replace({'JOSE IGNACIO TADEO': '21'}, regex=False)
data_final.Usuario = data_final.Usuario.replace({'JOSE IAN': '22'}, regex=False)
data_final.Usuario = data_final.Usuario.replace({'ASHLEY ': '23'}, regex=False)
data_final.Usuario = data_final.Usuario.replace({'JOSHUA': '24'}, regex=False)
data_final.Usuario = data_final.Usuario.replace({'YEREMI YAZMIN ': '25'}, regex=False)
data_final.Usuario = data_final.Usuario.replace({'MA DEL ROSARIO ': '26'}, regex=False)
data_final.Usuario = data_final.Usuario.replace({'BENJAMIN': '27'}, regex=False)
data_final.Usuario = data_final.Usuario.replace({'INGRID': '28'}, regex=False)
```

Calculamos matrices de correlación para identificar relaciones entre variables como botón correcto, color presionado, tiempo de interacción, y dificultad. Utilizamos `matshow()` de Matplotlib para representar estos valores visualmente. Gracias a estas visualizaciones, detectamos relaciones fuertes entre variables específicas que más adelante usamos para construir modelos predictivos.

```
fig, ax = plt.subplots(figsize=(20, 15))
cax = ax.matshow(corr_factors1, cmap="Blues")
fig.colorbar(cax)

# Añadir anotaciones manualmente
for i in range(corr_factors1.shape[0]):
    for j in range(corr_factors1.shape[1]):
        ax.text(j, i, f"{corr_factors1.iloc[i, j]:.2f}",
                ha="center", va="center", fontsize=20)

plt.xticks(range(len(corr_factors1.columns)), corr_factors1.columns, rotation=90, fontsize=12)
plt.yticks(range(len(corr_factors1.index)), corr_factors1.index, fontsize=12)
plt.savefig('General.png', dpi=300, bbox_inches='tight')
plt.show()
```

Python



Este análisis se repitió para distintos subconjuntos de datos por usuario, visualizando la relación entre variables como:

- Tiempo de interacción
- Color presionado
- Botón correcto
- Dificultad

Filtramos subconjuntos por usuario y modelamos sus datos de forma independiente. En cada caso, establecimos una variable dependiente (por ejemplo, tiempo de interacción) y una independiente (por ejemplo, número de interacción). Utilizamos la función `curve_fit()` para ajustar modelos cuadráticos o exponenciales que explicaran el comportamiento observado.

```
Vars_Indep= data_final[['mini juego']]
Var_Dep= data_final['Juego']

x = Vars_Indep
y = Var_Dep

def func1(x, a, b, c):
    return a*np.exp(-b*x) + c

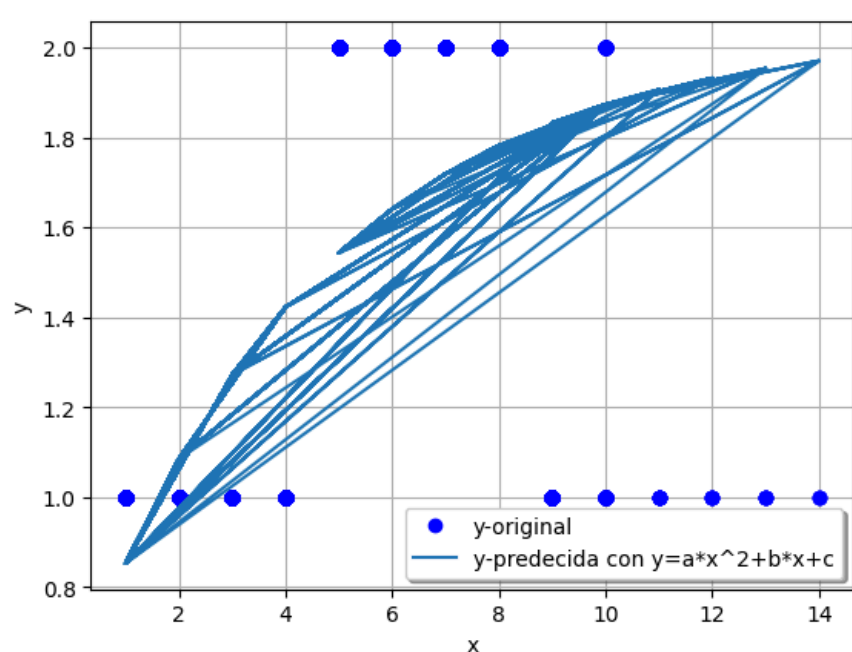
parametros, covs= curve_fit(func1, data_final['mini juego'], data_final['Juego'])
parametros
```

Estos modelos nos permitieron explorar cómo evoluciona el desempeño de los usuarios a medida que interactúan más con la plataforma. Observamos patrones crecientes, decrecientes o estables dependiendo del tipo de tarea y del usuario.

Para cada modelo ajustado, generamos gráficos con los datos reales (puntos azules) y la predicción del modelo (línea continua). Esto lo hicimos usando `plt.plot()`. Estas gráficas nos dieron una herramienta visual potente para evaluar si el modelo se adaptaba bien a los datos. También pudimos comparar el comportamiento de distintos usuarios entre sí.

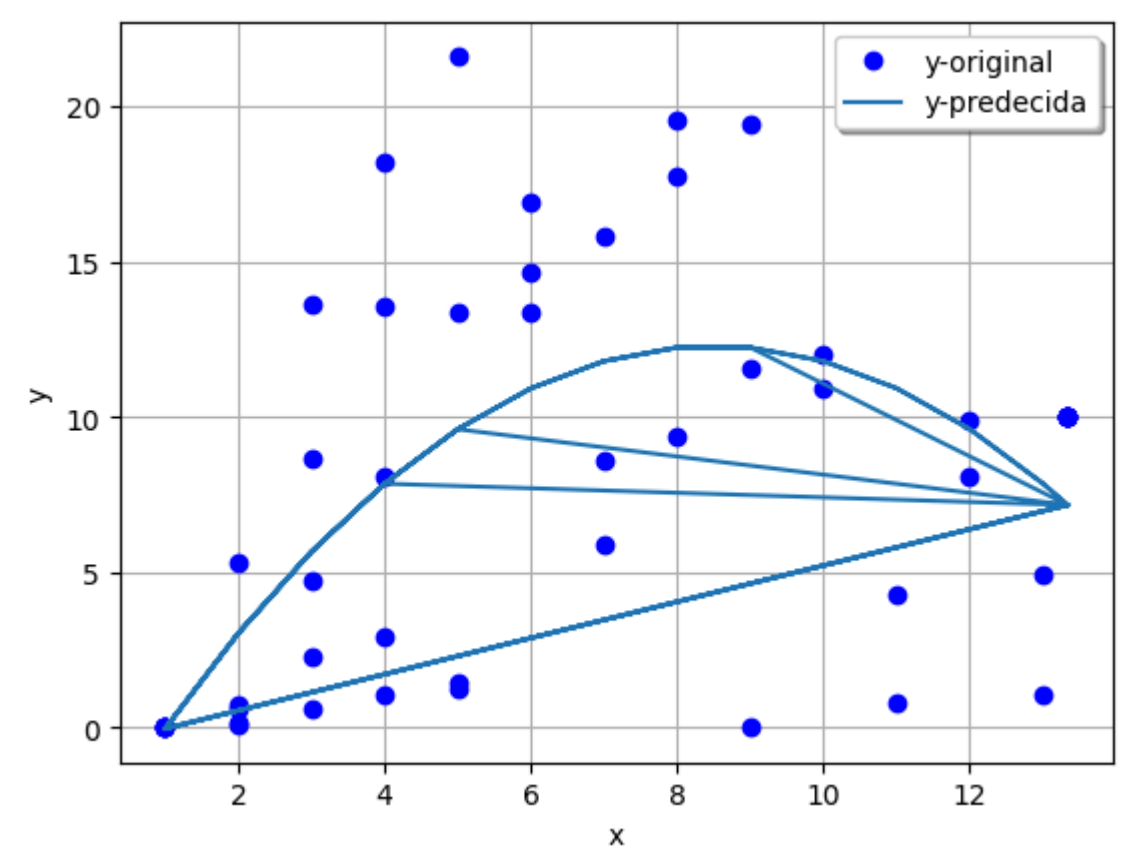
```
plt.plot(x, y, 'bo', label="y-original")
plt.plot(x, y_pred, label="y-predecida con  $y=a*x^2+b*x+c$ ")
plt.xlabel('x')
plt.ylabel('y')
plt.legend(loc='best', fancybox=True, shadow=True)
plt.grid(True)
plt.show()
```

Python



Se aplicó este procedimiento a combinaciones como:

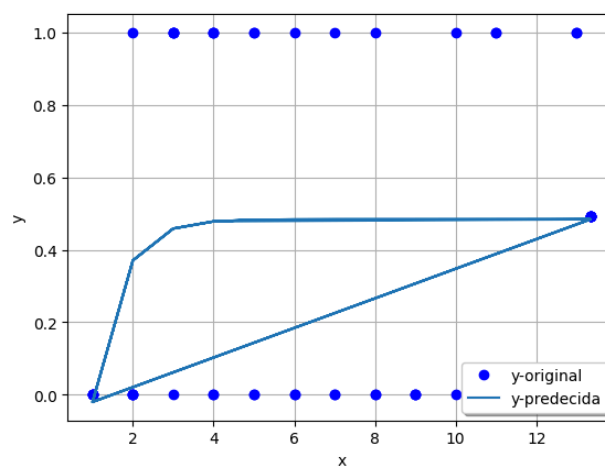
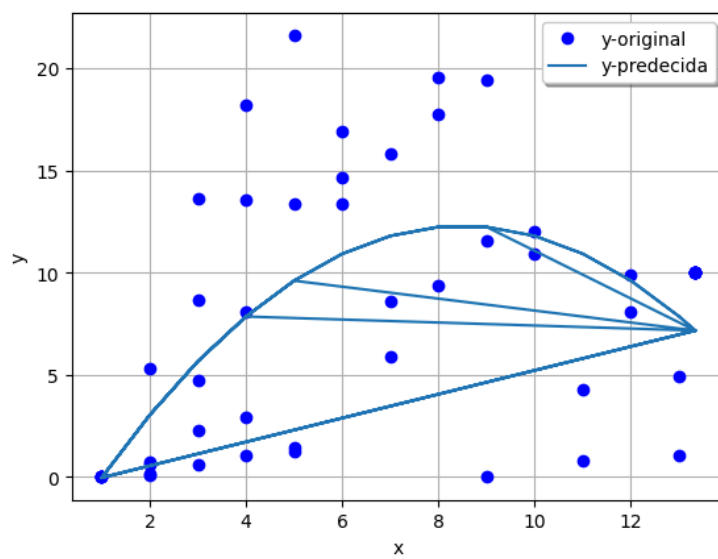
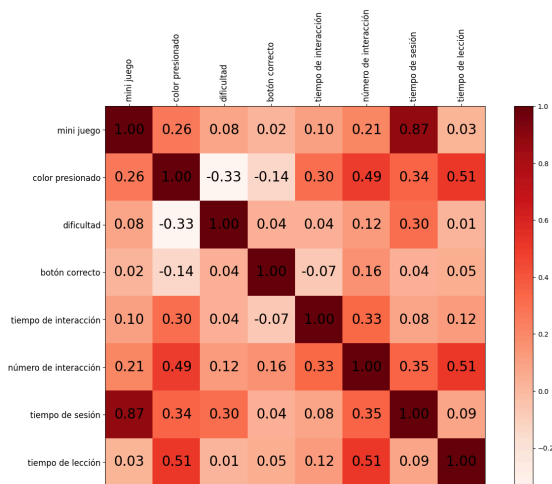
- Color presionado → botón correcto
- Tiempo de sesión → dificultad
- Número de interacción → tiempo de interacción

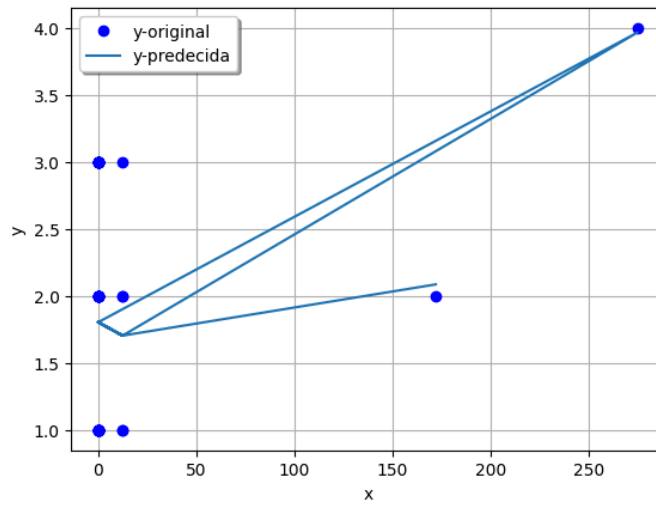
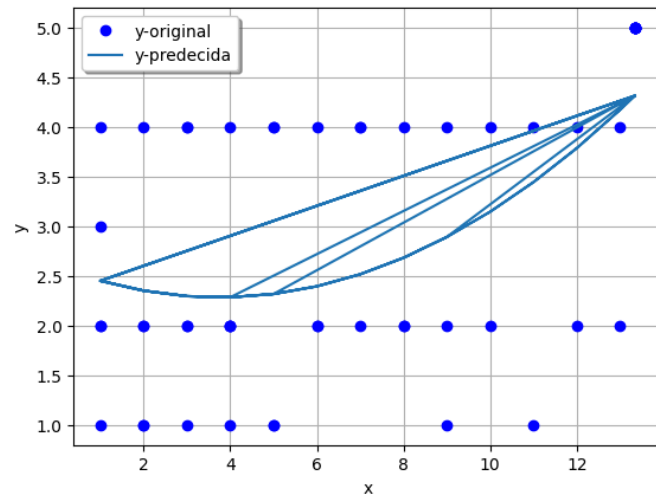


Cada uno de los modelos anteriores fue visualizado con gráficos de dispersión ('bo') y líneas de predicción. Estas gráficas permiten observar si la tendencia del usuario era creciente, decreciente o estable en variables como precisión, interacción o desempeño.

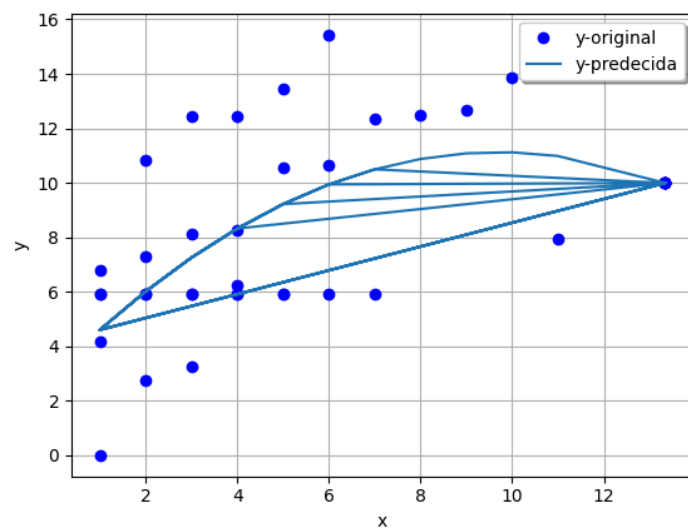
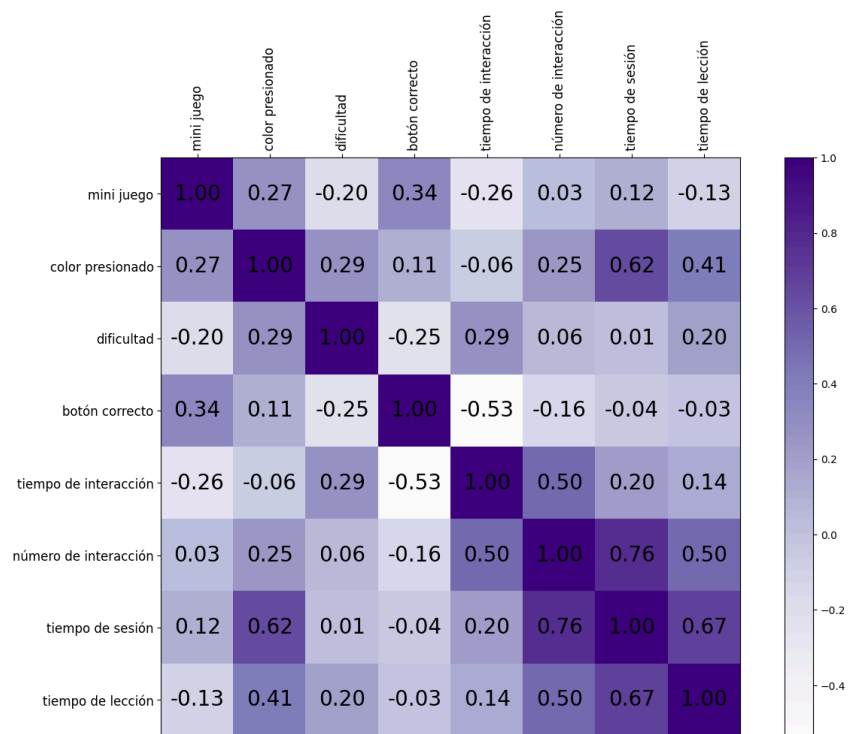
Finalmente tuvimos los resultados por usuario en orden de relacion (heatmap, tiempo de interaccion, boton correcto, color presionado)

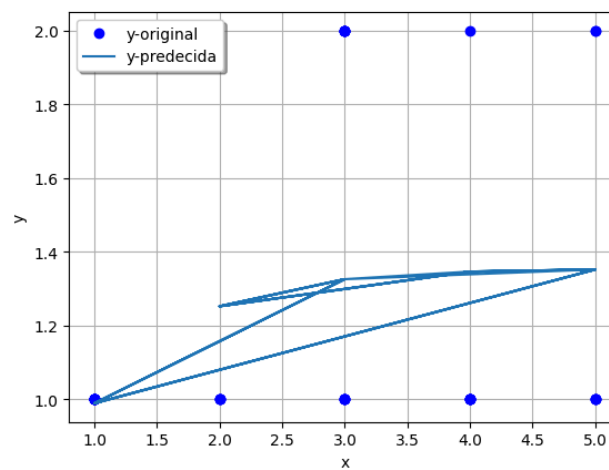
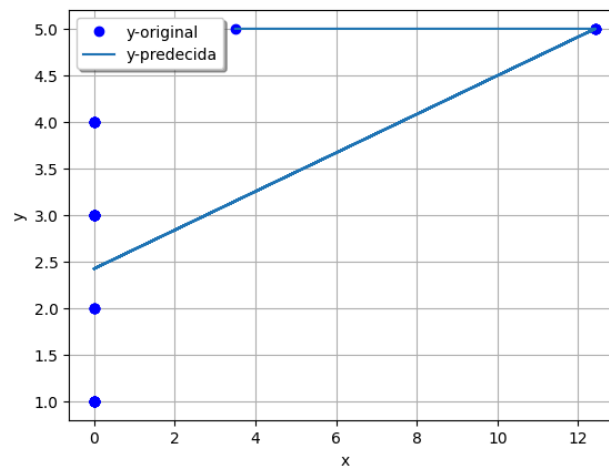
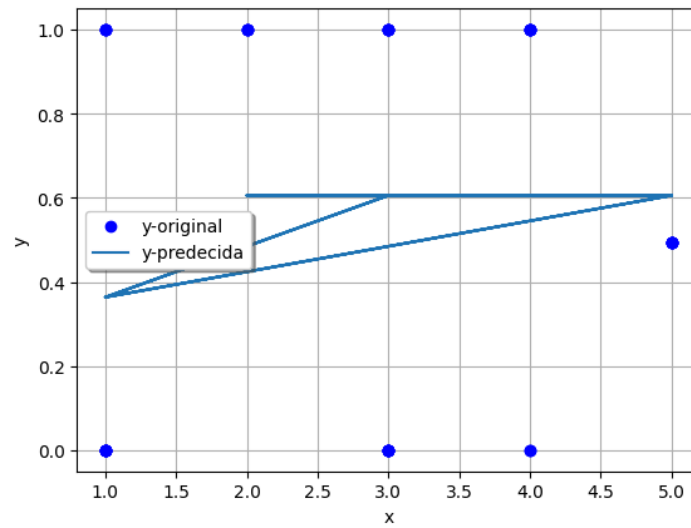
- Benjamin



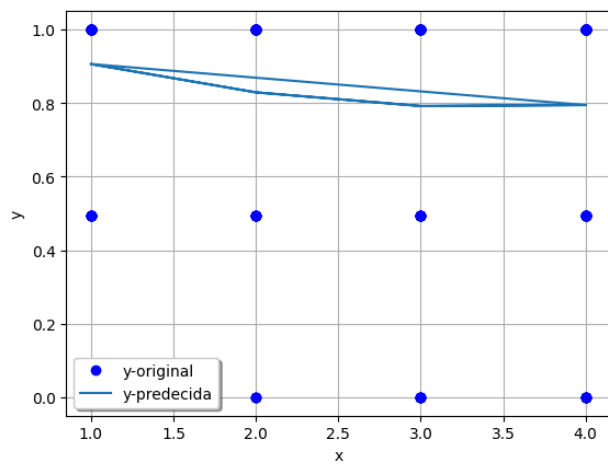
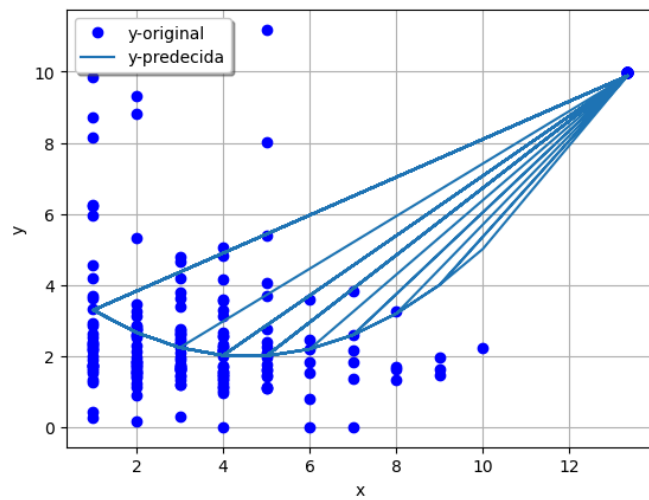
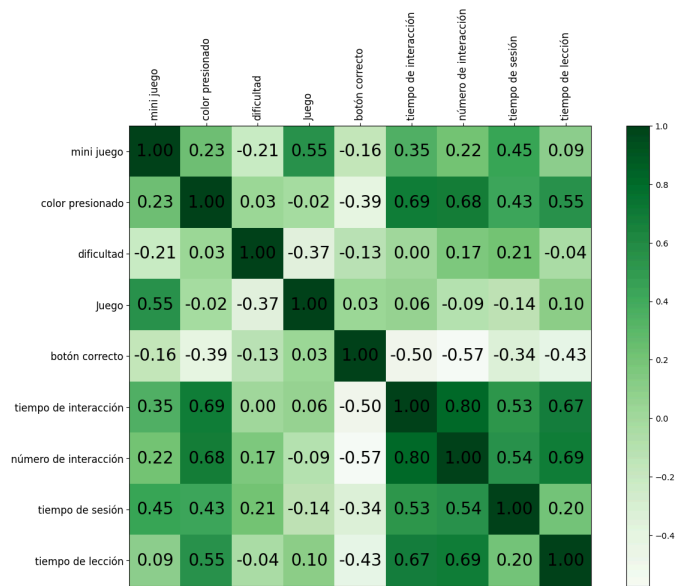


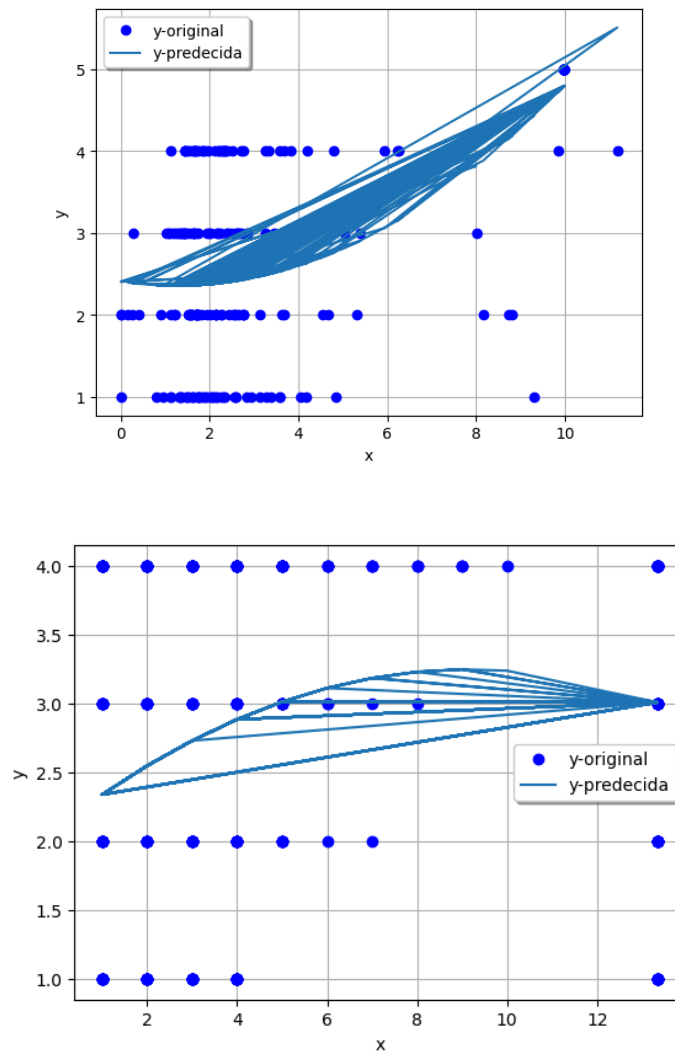
- Carlos Abel



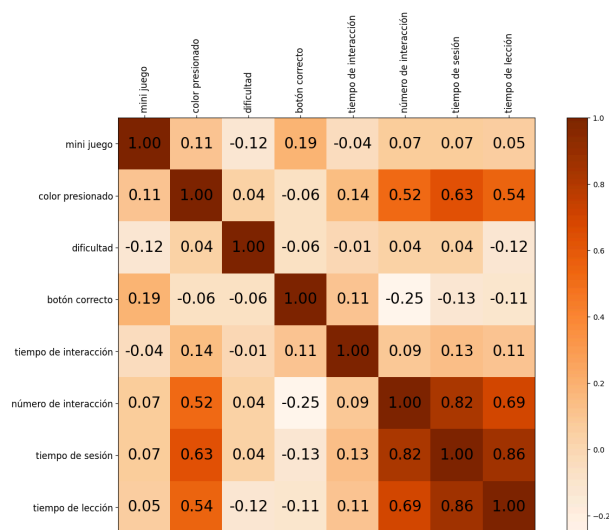


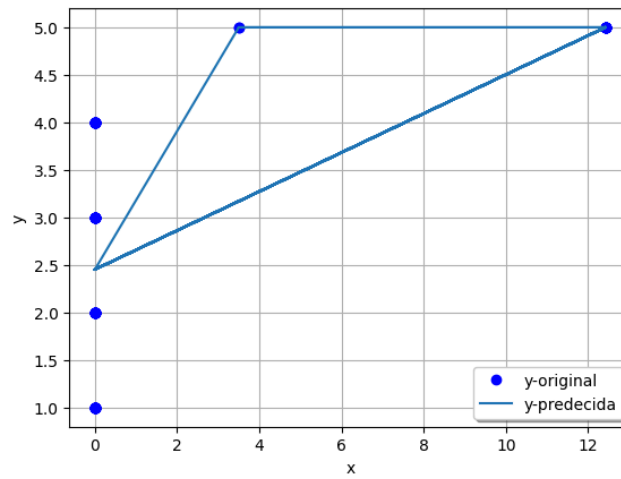
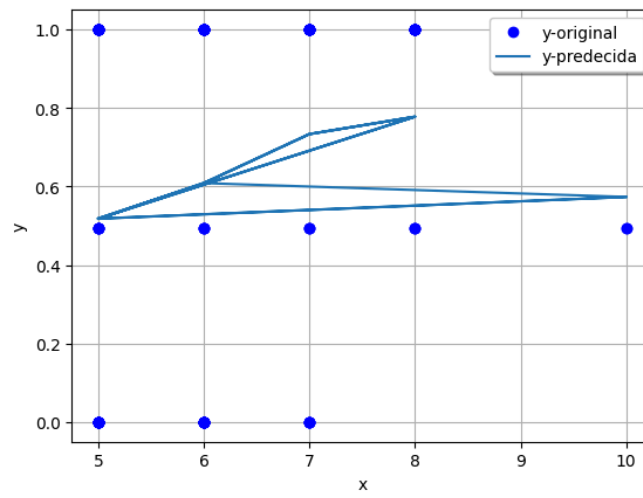
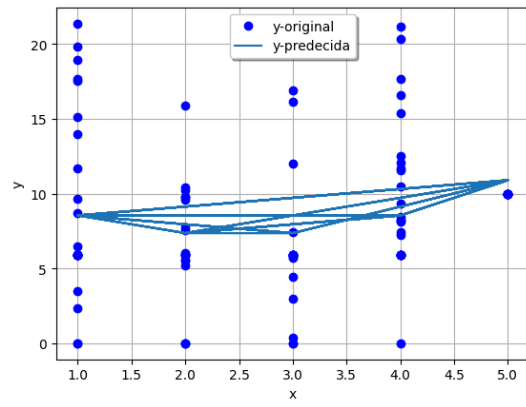
- Carlos Enrique

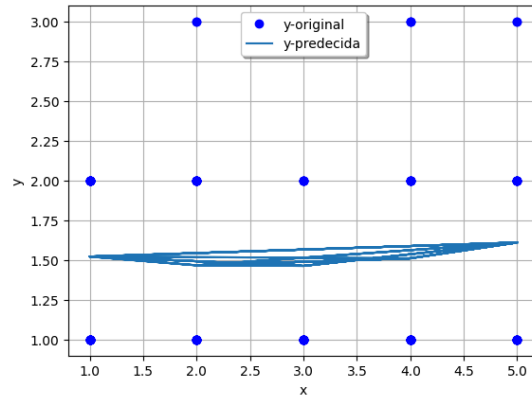




- Concepción







- Denisse

